YOLOv26 Model Architecture

For the implementation of real-time instance segmentation, the YOLOv26 model was chosen because of its capacity to provide pixel-level precision without reducing real-time inference speeds.

Backbone: The model uses a Cross-Stage Partial (CSP) network architecture designed specifically for the efficient extraction of hierarchical features of complex microscopic textures. The CSP network provides a lower computational load and high sensitivity towards morphological features, such as the circularity of prasinophytes and irregular shapes typical of AOM.

Neck (Feature Pyramid Network): This is included to facilitate multiscale feature fusion, which is vital because of the wide variety of organic particle sizes. The inclusion of the neck allows for the transfer of structural information combined with semantic information.

Architecture Head (NMS-free and Decoupled): In order to achieve faster inference speeds, the network head uses decoupling, meaning that classification, bounding box regression, and mask generation are carried out in parallel branches of the model. To achieve mask generation, a branch similar to ProtoNet calculates the prototypes of masks and generates the instance coefficients to sum up the individual instance masks ($M = \sigma(C \cdot P)$.

Uniform input image size: To ensure consistent image processing, all input images were processed using a common resolution of 640 × 640 pixels. This resolution ensures a balance between retaining morphological features to differentiate *Halosphaeropsis liassica* clusters from AOM granules and inference speeds.

Training Procedure

Model training was done with the help of the Python programming language and with the Ultralytics YOLOv26 framework through our scripts: ‘bora_toae_paly.ipynb’. Such training was provided due to the robust environment that allows to fine-tune instance segmentation model. The following parameters and strategies were used in order to achieve proper convergence and optimization of the model:

Hardware Specifications: Training was performed with the help of a Tesla T4 GPU which consists of 16 GB of GDDR6 memory and has 8.1 TFLOPS system RAM.

Learning rate: Learning rate value equals 0.001; AdamW optimizer with cosine annealing is used. Such configuration is aimed at performing big enough weight updates in the first stage of the training procedure and tuning of the weights in the further training stages to reach global minimum.

Batch size: Batch size is 16 in order to use GPU memory optimally (Tesla T4 GPU) while having a diverse number of palynofacies classes in one training epoch.

Epochs: Model training lasts for 100 epochs, 50 of which can be stopped early if the model stops improving its performance on the validation set. Training time was chosen so as to prevent overfitting on the special TOAE dataset and achieving stable performance on the validation set.

Caching (cache=True): Used in order to load the whole dataset to the RAM in order to make subsequent epochs to train faster.

Validation (val=True): Periodic testing on the unseen validation data in order to estimate current model performance.

Output formats (save_json=True, save_txt=True, plots=True): Saving of predictions in the JSON and text format as well as training plots.

In order to make model more robust and perform well in cases when the state of preservation differs in other basins, some data augmentation techniques were used with bora.py script:

Mosaic augmentation to combine four pictures into one. Random rotations in range of 360 degrees. Brightness and contrast set (+/- 0.2, p=0.6) and colors (p=0.4) to create effects to changing conditions such as lighting and thermal maturity. Blur set (not more than 3-5, p=0.3) imitates out of focus microscopy. Texture ser (p=0.2-0.3) for sharpen edges. Also ISO Noise set as (p=0.2) introduces typical sensor grain.

Evaluation Metrics

The performance of the model was comprehensively measured using metrics which extent of detection and instance segmentation.

Precision (π): Measures the ratio of correct palynofacies instances out of total predicted instances.

Recall (ρ): Measures the ratio of the correctly predicted instances out of total ground truth instances.

F-score: Measures the average between precision and recall. These are the measures that measure the ability of the model to minimize false negatives and false positives respectively.

Mean Average Precision (mAP@ and mAP@-0.5) measures AP for each class at different thresholds for IoU. Mean Average Precision (mAP@0.5) is especially important for instance segmentation because of the requirement of high overlap of pixels.

Confusion matrices includes True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN) for each class. It is generated for each class to show details about the classification performance of model.

Software Architecture and Data Pipeline

The implementation of our APAQ method in 2 phases, within a modular Python-based software ecosystem as follows: (1) Deep Learning Inference; and (2) Automated Quantitative Analysis and Visualisation (Table 1).

Phase 1: Deep Learning Inference (bora_toae_paly.ipynb) and data augmentation script (bora.py): Acting for training phase.

Phase 2: Quantitative Analysis and Visualisation (palynofacies_full.py): It takes action after the training phase once the model is ready for inference. Acts as the core analysis engine, performing automated extraction of morphological and spectral features from the deep learning model inference output.

Pixel-to-Micron Calibration: The incorporation of a hardware dependent calibration process ensures cross-platform generalizability and hardware compatibility during analysis through the use of an analytical engine to calculate absolute areas in μm^2. In particular, the deep learning model operates on a standard image input size of 640 x 640 pixels to ensure that features can be detected in a scale independent manner, while the post-processing engine ‘microscope.py’ performs real-time coordinate transformation based on the source used in image acquisition. Notably, ‘microscope.py’ script is implemented in the latter process, which allows independent scaling constants based on µm/pixel ratio values for each microscope wishing to analyze images from (e.g., Olympus BX51/Zeiss Axiocam configuration: 1.2403 µm/pixel at 10x magnification, 20x : 0.6202, 40x : 0.3101 µm/pixel).

Relative Abundance: The script calculates the sum total area of each class as well as counts each image separately while returning percentages of both.

Thermal Alteration Index (TAI): Engine as a novel capability of the script enables the automatic estimation of Thermal Alteration Index (TAI) from extracted HSV (Hue, Saturation, Value) profiles of segmented palynomorphs. It calculates the average Hue and Value of the pixels in the segment mask. Low Value (V) and darker Hues correspond to higher thermal maturity and are defined as being within "Immature" range (Yellow), and going up to "Overmature" range (Black/dead Carbon). Automatic Reporting: The script produces a palynofacies_report.xlsx report file that includes image-based and statistical data, together with TAI assessments along with `summary_charts.png`.

Paleoenvironmental Classification (tysonplot.py): This script is set to perform geological classification of samples using the standard Ternary Diagram introduced by Tyson (1995). It determines the exact pixel coordinates for the Tyson facies fields and automatically aggregates the classified sub-facies classes accordingly to the 3 Tyson poles. Ultimately, providing a direct visual classification of basin proximity and redox conditions.

Script	**Primary Function**	**Key Output**
bora.py	Data Augmentation	Augmented Image/Labeled Dataset
bora_toae_paly.ipynb	Model Training	model.pt (Trained Weights)
palynofacies_full.py	Quantification & TAI & Relative Abundance	palynofacies_results.csv, relative_abundance_log, tai_diagrams.png, summary_charts.png
tysonplot.py	Facies Mapping	tyson_ternary_plot.png

Table 1 – Phases of software architecture and data pipeline